Experiment 2 — NumPy Operations

PIN: `241UDS0096`

Aim

To practice NumPy array creation, attributes, indexing, slicing, broadcasting, arithmetic, statistical operations, concatenation, reshaping, sorting, splitting, and selecting data.

Step 1 — Install and Import NumPy

Import NumPy using the conventional alias `np`. NumPy is usually already available in Jupyter; if not, run `!pip install numpy` in a separate cell.

In [43]:
```
# PIN-specific experiment variables use the suffix 241UDS0096
import numpy as np
```

Step 2 — Create Arrays for Three Transit Services

Each array contains bookings for Week 1 through Week 4. The values are also stored in the companion CSV file.

In [44]:
```
metro_bookings_241UDS0096 = np.array([720, 810, 790, 860])
citybus_bookings_241UDS0096 = np.array([560, 610, 640, 700])
rapidride_bookings_241UDS0096 = np.array([340, 390, 370, 430])

metro_bookings_241UDS0096, citybus_bookings_241UDS0096,
rapidride_bookings_241UDS0096
```

Out[44]:
```
(array([720, 810, 790, 860]),
 array([560, 610, 640, 700]),
 array([340, 390, 370, 430]))
```

Step 3 — Check Array Properties

Inspect dimensionality, shape, number of elements, data type, and bytes used by each element.

In [45]:
```
metro_bookings_241UDS0096.ndim
```

Out[45]:
```
1
```

In [46]:
```
citybus_bookings_241UDS0096.shape
```

Out[46]:
```
(4,)
```

In [47]:
```
rapidride_bookings_241UDS0096.size
```

Out[47]:
```
4
```

In [48]:
```
metro_bookings_241UDS0096.dtype
```

Out[48]:
```
dtype('int64')
```

In [49]:
```
metro_bookings_241UDS0096.itemsize, citybus_bookings_241UDS0096.itemsize,
rapidride_bookings_241UDS0096.itemsize
```

Out[49]:
```
(8, 8, 8)
```

Step 4 — Retrieve Data Using Indexing and Slicing

Use an index to select one value, `np.unique` to list distinct values, and slices to select ranges of weeks.

In [50]:
```
rapidride_bookings_241UDS0096[0]
```

Out[50]:
```
np.int64(340)
```

In [51]:
```
np.unique(citybus_bookings_241UDS0096)
```

Out[51]:
```
array([560, 610, 640, 700])
```

In [52]:
```
metro_bookings_241UDS0096[0:2]
```

Out[52]:
```
array([720, 810])
```

In [53]:
```
citybus_bookings_241UDS0096[2:]
```

Out[53]:
```
array([640, 700])
```

In [54]:
```
rapidride_bookings_241UDS0096[-3:]
```

Out[54]:
```
array([390, 370, 430])
```

Step 5 — Assign New Values Using Broadcasting

Copy the Metro array, append two extra values, then replace those appended values using slice assignment. The original array remains unchanged.

In [55]:
```
metro_bookings_241UDS0096
```

Out[55]:
```
array([720, 810, 790, 860])
```

In [56]:
```
metro_bookings_new_241UDS0096 = metro_bookings_241UDS0096.copy()
metro_bookings_new_241UDS0096
```

Out[56]:
```
array([720, 810, 790, 860])
```

In [57]:
```
metro_bookings_new_241UDS0096 = np.append(metro_bookings_new_241UDS0096,
[900, 880])
metro_bookings_new_241UDS0096
```

Out[57]:
```
array([720, 810, 790, 860, 900, 880])
```

In [58]:
```
metro_bookings_new_241UDS0096[4:6] = [920, 890]
metro_bookings_new_241UDS0096
```

Out[58]:
```
array([720, 810, 790, 860, 920, 890])
```

Step 6 — Arithmetic Operations

Perform scalar addition, subtraction, multiplication, and division on selected booking counts.

In [59]:
```
rapidride_bookings_241UDS0096
```

Out[59]:
```
array([340, 390, 370, 430])
```

In [60]:
```
rapidride_bookings_241UDS0096 + 25
```

Out[60]:
```
array([365, 415, 395, 455])
```

In [61]:
```
rapidride_bookings_241UDS0096[3] - 15
```

Out[61]:
```
np.int64(415)
```

In [62]:
```
rapidride_bookings_241UDS0096[0] * 2
```

Out[62]:
```
np.int64(680)
```

In [63]:
```
rapidride_bookings_241UDS0096[1:3] / 2
```

Out[63]:
```
array([195., 185.])
```

Step 7 — Statistical Operations

Calculate the total and minimum/maximum Metro bookings, and the mean for each service.

In [64]:
```
metro_bookings_241UDS0096, citybus_bookings_241UDS0096,
rapidride_bookings_241UDS0096
```

Out[64]:
```
(array([720, 810, 790, 860]),
 array([560, 610, 640, 700]),
 array([340, 390, 370, 430]))
```

In [65]:
```
np.sum(metro_bookings_241UDS0096)
```

Out[65]:
```
np.int64(3180)
```

In [66]:
```
np.max(metro_bookings_241UDS0096), np.min(metro_bookings_241UDS0096)
```

Out[66]:
```
(np.int64(860), np.int64(720))
```

In [67]:
```
print(
    np.mean(metro_bookings_241UDS0096),
    np.mean(citybus_bookings_241UDS0096),
    np.mean(rapidride_bookings_241UDS0096)
)
```

```
795.0 627.5 382.5
```

Step 8 — Combine All Service Data

Confirm that the three arrays are one-dimensional, concatenate them into one array, and calculate the combined total.

In [68]:
```
metro_bookings_241UDS0096.ndim, citybus_bookings_241UDS0096.ndim,
rapidride_bookings_241UDS0096.ndim
```

Out[68]:
```
(1, 1, 1)
```

In [69]:
```
all_service_bookings_241UDS0096 = np.concatenate([
    metro_bookings_241UDS0096,
    citybus_bookings_241UDS0096,
    rapidride_bookings_241UDS0096
])
all_service_bookings_241UDS0096
```

Out[69]:
```
array([720, 810, 790, 860, 560, 610, 640, 700, 340, 390, 370, 430])
```

In [70]:
```
np.sum(all_service_bookings_241UDS0096)
```

Out[70]:
```
np.int64(7220)
```

Step 9 — Reshape into a 3 × 4 Matrix

Check the one-dimensional array's shape, reshape it into three rows and four columns, then inspect the new dimensions.

In [71]:
```
all_service_bookings_241UDS0096.ndim, all_service_bookings_241UDS0096.shape
```

Out[71]:
```
(1, (12,))
```

In [72]:
```
all_service_bookings_241UDS0096 = all_service_bookings_241UDS0096.reshape(3,
4)
all_service_bookings_241UDS0096
```

Out[72]:
```
array([[720, 810, 790, 860],
       [560, 610, 640, 700],
       [340, 390, 370, 430]])
```

In [73]:
```
all_service_bookings_241UDS0096.ndim, all_service_bookings_241UDS0096.shape
```

Out[73]:
```
(2, (3, 4))
```

Step 10 — Sort the Array

View the first service row, then sort each row and each column independently. Sorting returns sorted values; it does not change the original array unless assigned.

In [74]:
```
all_service_bookings_241UDS0096
```

Out[74]:
```
array([[720, 810, 790, 860],
       [560, 610, 640, 700],
       [340, 390, 370, 430]])
```

In [75]:
```
all_service_bookings_241UDS0096[0]
```

Out[75]:
```
array([720, 810, 790, 860])
```

In [76]:
```
np.sort(all_service_bookings_241UDS0096, axis=1)
```

Out[76]:
```
array([[720, 790, 810, 860],
       [560, 610, 640, 700],
       [340, 370, 390, 430]])
```

In [77]:
```
np.sort(all_service_bookings_241UDS0096, axis=0)
```

Out[77]:
```
array([[340, 390, 370, 430],
       [560, 610, 640, 700],
       [720, 810, 790, 860]])
```

Step 11 — Split the Array

Split the matrix along its rows into separate arrays for Metro, CityBus, and RapidRide.

In [78]:
```
split_metro_241UDS0096, split_citybus_241UDS0096, split_rapidride_241UDS0096
= np.split(
    all_service_bookings_241UDS0096,
    [1, 2],
    axis=0
)
```

In [79]:
```
split_metro_241UDS0096
```

Out[79]:
```
array([[720, 810, 790, 860]])
```

In [80]:
```
split_citybus_241UDS0096
```

Out[80]:
```
array([[560, 610, 640, 700]])
```

In [81]:
```
split_rapidride_241UDS0096
```

Out[81]:
```
array([[340, 390, 370, 430]])
```

Step 12 — Retrieve Week 2 Bookings of All Services

Select the second column for all three rows. The result remains a two-dimensional column array.

In [82]:
```
all_service_bookings_241UDS0096
```

Out[82]:
```
array([[720, 810, 790, 860],
       [560, 610, 640, 700],
       [340, 390, 370, 430]])
```

In [83]:
```
week_2_bookings_241UDS0096 = all_service_bookings_241UDS0096[:3, 1:2]
print(week_2_bookings_241UDS0096)
```

```
[[810]
 [610]
 [390]]
```

Result

NumPy operations were practiced using the weekly transit-bookings dataset: array creation and properties, indexing and slicing, appending and slice assignment, arithmetic, statistics, concatenation, reshaping, sorting, splitting, and selecting Week 2 values.